

ECDAT

Enterprise Cryptographic Discovery & Analysis Tool

Complete Project Description, Implementation Plan & Experimental Features

Core lifecycle: Discover → Evidence → Correlate → Assess → Plan → Migrate → Re-scan → Verify
Experimental extensions: Runtime Crypto Tracing • eBPF/Uprobes • Auto-Patch + Validation • Custom Crypto Detection • Passive PCAP Discovery • Quantum Resource Estimation

1. Executive Summary

ECDAT is an evidence-driven enterprise cryptographic inventory and post-quantum migration platform. It discovers cryptographic assets across source code, dependencies, binaries/firmware, certificates and authorized network endpoints; preserves evidence and provenance; correlates observations into a Crypto Asset Graph; separates classical weakness from quantum exposure; assesses migration impact; generates dependency-aware migration plans; and verifies the post-migration state through re-scanning.

The project should not be positioned as a generic vulnerability scanner or penetration-testing platform. Its purpose is to answer: **What cryptography exists, where is it used, what evidence proves it, what depends on it, what risk does it create, what should change, and did the change actually happen?**

2. Core Problem

- Cryptography is distributed across applications, source code, libraries, binaries, firmware, certificates, network services and data flows.
- Different scanners can produce isolated findings without a shared identity or evidence model.
- Network inspection sees cryptography exposed at the network boundary but cannot establish every internal application-level cryptographic operation.
- Static analysis can miss dynamic dispatch, custom implementations and some obfuscated or compiler-transformed patterns.
- Quantum migration is dependency-sensitive and requires prioritization rather than a flat replacement list.
- Migration completion should be demonstrated with new evidence rather than a manually checked box.

3. Core ECDAT Architecture



4. Discovery Surfaces

Surface	Purpose
Source code	Detect crypto APIs, imports, constants, aliases and implementation indicators; prioritize C/C++
Dependencies	Identify libraries/components and relationships that affect cryptographic migration.
Binary / firmware	Static analysis of ELF/PE structure, sections, entropy and cryptographic signatures/indicators
Network / TLS	Authorized observation of TLS versions, cipher/key-exchange behavior, certificates and PQC

Surface	Purpose
Certificates	Capture certificate cryptographic metadata and relationships.
Runtime (experimental)	Observe cryptographic calls made by an executable while it actually runs.
Passive PCAP (experimental)	Extract cryptographic network evidence from an authorized packet capture without active en

5. Unified Evidence Model

Every finding should use one common record format so that all discovery engines can feed the same correlation and graph pipeline.

EvidenceRecord
asset_id • asset_type • algorithm • cryptographic_role • source_surface • evidence_type • location/endpoint • confidence • timestamp • input_hash • engine_version

- Static evidence: inferred from source, configuration or binary analysis.
- Runtime evidence: observed during actual execution.
- Network evidence: observed during authorized protocol interaction or passive capture.
- Corroborated evidence: supported by multiple independent discovery surfaces.
- Unknown: a property that was not measured remains unknown; ECDAT should not guess.

6. Crypto Asset Graph

The graph is the central integration layer. Nodes represent assets such as applications, libraries, algorithms, certificates, endpoints, binaries and data flows. Edges represent relationships such as uses, implements, depends-on, protects, serves and authenticated-by.

APPLICATION → USES → LIBRARY → IMPLEMENTS → ALGORITHM
APPLICATION → SERVES → ENDPOINT → USES → TLS CRYPTO
ENDPOINT → AUTHENTICATED BY → CERTIFICATE
CRYPTO ASSET → PROTECTS → DATA FLOW

A practical first implementation can use a lightweight graph library and deterministic identity/correlation heuristics. A full graph database is not required for the prototype.

7. Network Scanner

The current network scanner is a cryptographic observation engine. It performs authorized TLS interaction and records visible network cryptographic posture, including TLS behavior, cipher/key-exchange information, certificate information and PQC/hybrid probes where supported.

Example: a network scan may report **TLS_AES_128_GCM_SHA256**. That is the negotiated TLS cipher suite; SHA-256 in the suite name is a hash component, not a statement that SHA-256 is encrypting the application data.

The network scanner should not be expected to reveal arbitrary cryptography performed internally by a remote application behind TLS. That limitation is why ECDAT combines network evidence with source, binary and runtime evidence.

8. Classical + Quantum Risk

Use deterministic, auditable rules for the core risk engine. Classical weakness can be handled with a maintained algorithm/status table, while quantum planning can use a Mosca-style relationship between data/security lifetime, migration time and a quantum-threat horizon.

- Classical examples: MD5, SHA-1, DES and RC4 can be mapped to documented weakness/deprecation status.
- Quantum exposure: relevant public-key mechanisms can be flagged for PQC migration planning.
- PQC reference mapping should be deterministic, such as FIPS 203 ML-KEM, FIPS 204 ML-DSA and FIPS 205 SLH-DSA, rather than an LLM inventing replacements.
- Do not present a quantum-hardware forecast as a certainty.

9. Migration Planner

Migration should be generated from evidence plus graph relationships. The planner should identify affected components, dependencies, migration actions, validation criteria and rollback considerations.

```
INVENTORY → DEPENDENCY CHECK → LIBRARY/HOSTING CHANGE → CRYPTO CHANGE → STAGING → VALIDATION → CUTOVER →  
ROLLBACK → RE-SCAN
```

Migration estimates should be labeled as heuristic unless measured or supplied by a vendor/system owner.

10. Re-scan / Verification — Core Differentiator

After a migration, ECDAT should scan again and compare the resulting evidence against the baseline.

```
BEFORE: RSA-2048 observed; PQC/hybrid not observed  
AFTER:  RSA-2048 no longer observed; PQC/hybrid observed  
RESULT: VERIFIED / NOT VERIFIED / INCONCLUSIVE
```

This creates an auditable closed loop: the system does not equate a checked migration task with proof that the deployed cryptographic state changed.

11. CBOM

ECDAT should export a machine-readable Cryptography Bill of Materials using a valid CycloneDX representation. Each asset can retain source surface, evidence type, confidence and provenance metadata. CBOM export should be schema-validated rather than hand-waved.

12. Experimental A — Runtime Crypto Tracing

Goal: observe cryptographic operations that actually execute, rather than only inferring them from source or binary artifacts.

For Linux-owned services/binaries, a runtime instrumentation layer can observe calls to cryptographic libraries during a controlled test run. A first prototype can use an **LD_PRELOAD** shim around selected OpenSSL/libcrypto APIs; an advanced implementation can use eBPF uprobes against an already-running process.

```
Application executes → crypto library call occurs → interceptor records function/algorithm-relevant  
metadata → JSON evidence stream → ECDAT EvidenceRecord → graph
```

Scope: systems the organization is authorized to execute/instrument. This is not a mechanism for remotely observing arbitrary third-party websites.

13. Experimental B — eBPF / Uprobe Runtime Discovery

eBPF uprobes can attach to user-space functions in a running Linux process and observe selected crypto-library calls without modifying the application binary. This is potentially the enterprise-realistic version of runtime discovery because the target service can remain running.

- Attach to selected symbols such as relevant OpenSSL/libcrypto functions where symbols are available.
- Record process identity, executable/library identity, function and safe algorithm/key-size metadata where available.
- Convert events into the common evidence schema.
- Correlate runtime observations with static findings using executable/library hashes, process identity, deployment metadata or other explicit evidence.
- Report unsupported/unobservable functions as unknown rather than assuming absence.

Risks: Linux-specific implementation, privileges/capabilities, kernel support, symbol availability, stripped/static binaries and instrumentation overhead. Keep this experimental and clearly label coverage.

14. Experimental C — Auto-Generated Migration Patches

Instead of stopping at a recommendation, ECDAT can support a narrow set of safe, pre-defined transformation templates. The key is limited scope, not unrestricted AI-generated code modification.

Detect known weak pattern → generate predefined patch → generate tests → run tests → show exact diff → re-scan resulting code

- Start with 2–3 tightly controlled patterns.
- Use deterministic templates, not unrestricted LLM rewriting.
- Generate regression/differential tests around affected behavior.
- Require human review before applying a patch to real software.
- Do not claim mathematical proof of equivalence unless such a proof actually exists.

15. Experimental D — LLM-Assisted Custom Crypto Detection

Static signature/API detection can miss hand-rolled cryptographic implementations. An experimental semantic detector can flag suspicious code regions for further review.

source candidate → context/features → LLM/classifier assessment → explanation/evidence → human-review status → optional runtime correlation

- Use it as a secondary detector, not authoritative proof.
- Maintain a small evaluation set containing true crypto and crypto-looking non-crypto code.
- Track false positives and false negatives.
- Label LLM output as inference and keep deterministic scanner evidence separate.

16. Experimental E — Passive Network Discovery via PCAP

A passive network mode can process an authorized packet capture and extract TLS handshake information across many connections without actively probing each endpoint.

PCAP → TLS handshake parsing → observed versions/cipher suites/fingerprints → normalized evidence → endpoint/service correlation → inventory

- Use only an explicitly authorized capture.
- Passive observation can be less complete than active negotiation.
- ClientHello fingerprints can provide useful visibility but do not prove every server-side cryptographic capability.
- Do not treat a fingerprint as proof of a negotiated cipher unless the relevant handshake evidence supports that conclusion.

17. Experimental F — Quantum Resource Estimation

An optional research feature can estimate the computational resources required for quantum attacks against specific public-key systems using published resource-estimation methods.

This should not output fake precision such as 'RSA-2048 breaks in exactly N years.' A defensible output should expose assumptions, model, cited resource estimates and uncertainty.

- Use published Shor/resource-estimation literature.
- Separate theoretical resource estimates from current hardware capability.
- Keep the Mosca-style planning model as the primary migration prioritization mechanism.
- Only expose numerical forecasts if the team can defend the assumptions in technical Q&A.;

18. Experimental G — Binary / Firmware Analysis

Binary crypto discovery remains a difficult technical problem, especially for custom implementations, optimization differences and obfuscation. ECDAT can expose this as an experimental detection layer rather than claiming perfect detection.

- Current static layer: format/section/entropy/signature indicators.
- Future ML layer: opcode/CFG/IR features trained and evaluated across compiler optimization levels.

- Deep reverse engineering: optional Ghidra/angr/CPG integration.
- Every binary finding should expose its detection technique and confidence.

19. Cross-Surface Correlation Strategy

This is one of the hardest engineering problems in the system. ECDAT should not assume that two observations with the same algorithm name are the same asset.

Recommended identity hierarchy:

1. Exact known asset/deployment identifier
2. Same input/binary hash + location
3. Same application/library + version + deployment context
4. Same endpoint/service + certificate/deployment evidence
5. Heuristic relationship only → mark as inferred, not proven

Every graph edge should carry relationship evidence and confidence. A graph relationship should never silently become a fact merely because two records look similar.

20. Offline / Air-Gapped Architecture

ECDAT should support an offline-first mode suitable for sensitive environments.

- No telemetry required for core scanning.
- Local processing of uploaded source/binaries/captures.
- Local deterministic risk and migration tables.
- Optional knowledge updates can be imported as signed/versioned data rather than requiring live network access.
- Network scanning is restricted to explicitly authorized targets.

21. Standards / Regulatory Mapping

A reporting layer can map findings to applicable standards and organizational policy. Keep mappings explicit and versioned.

- NIST PQC standards: FIPS 203 / ML-KEM, FIPS 204 / ML-DSA, FIPS 205 / SLH-DSA.
- CycloneDX CBOM representation.
- Relevant organizational policies and applicable regulatory controls.
- Where a deadline or requirement is cited, record jurisdiction, system population, date/version and source.

22. Current Repository → Target Architecture

Existing / New component	Target role
source_scan.py	Static source discovery → EvidenceRecord
binary_scanner.py / binary_deep.py	Binary/firmware discovery → EvidenceRecord
network_prober.py	TLS/network observation → EvidenceRecord
pqc_probe.py	PQC/hybrid capability evidence
quantum_risk.py	Deterministic classical/quantum risk
knowledge_graph.py	Crypto Asset Graph
migration_planner.py	Dependency-aware migration plan
migration_simulator.py	Staging/simulation
cbom_generator.py	CycloneDX CBOM
scan_store.py	Evidence/baseline persistence

Existing / New component	Target role
agility_engine.py	Crypto-agility assessment
NEW: evidence model	Common schema and provenance
NEW: correlator	Identity resolution and graph edges
NEW: verifier	Before/after evidence diff
NEW: runtime tracer	Runtime crypto evidence — experimental
NEW: PCAP engine	Passive network evidence — experimental

23. Implementation Plan — Priority Order

Priority	Build
P0	Unified EvidenceRecord + provenance + explicit unknown state
P0	Normalize scanner outputs into one evidence pipeline
P0	Stable asset identity + correlation rules
P0	Crypto Asset Graph with evidence-backed edges
P0	Before/after baseline + re-scan verifier
P1	Risk + blast-radius + crypto-agility integration
P1	Dependency-aware migration planner
P1	CBOM evidence/provenance enrichment
P1	C/C++ source detection and strong source coverage
P2	Runtime LD_PRELOAD tracer
P2	eBPF/uProbe runtime tracing
P2	Passive PCAP discovery
P2	LLM-assisted custom-crypto detector
P3	Auto-patch + generated regression tests
P3	Binary ML classifier
P3	Quantum resource-estimation research module
P3	Ghidra/angr/CPG, SSH/QUIC/STARTTLS and deeper PKI

24. Recommended SIH Demo

The main demo should emphasize the implemented core rather than trying to show every experimental feature.

1. Scan a controlled C/C++ or mixed-language sample containing weak and modern crypto.
2. Show evidence with exact source locations and confidence.
3. Export CycloneDX CBOM.
4. Run deterministic classical + quantum risk analysis.
5. Show the Crypto Asset Graph and affected relationships.
6. Generate a dependency-aware migration plan.
7. Apply a controlled migration/simulation.
8. Re-scan.
9. Show the before/after evidence diff and VERIFIED/NOT VERIFIED/INCONCLUSIVE result.

If runtime tracing is stable, add it as the standout experimental demo: **static finding** → **execute owned test binary** → **runtime evidence** → **graph correlation** → **verification**.

25. Differentiation

- Closed-loop migration verification rather than stopping at inventory and recommendations.
- Evidence/provenance attached to findings and graph relationships.
- Cross-surface correlation instead of independent scanner lists.
- Runtime observation as an experimental discovery surface complementing static inference.
- Explicit distinction between observed, inferred and unknown properties.
- Dependency-aware migration context and blast-radius reasoning.
- Offline-first architecture suitable for sensitive environments.
- Machine-readable CBOM output.

These are engineering differentiators, not a claim that no existing commercial or research system contains any individual component.

26. What ECDAT Should Not Claim

- Cryptographic discovery itself is not being claimed as new.
- The inability of an external network scanner to see internal application crypto is not claimed as a new invention.
- The custom-crypto detector is not authoritative without measured validation.
- Binary detection is not claimed to be solved if it is signature-based.
- An LLM recommendation is not a proof of cryptographic correctness.
- A quantum break date is not a certainty.
- Advanced experimental modules are not claimed as implemented until actually tested.

27. Final System Definition

ECDAT is an evidence-driven cryptographic inventory and post-quantum migration platform that combines multiple discovery surfaces, evidence provenance, cross-surface correlation, a Crypto Asset Graph, deterministic risk analysis, dependency-aware migration planning, and re-scan verification.

Its experimental roadmap extends this foundation with runtime crypto tracing, eBPF/uProbe observation, passive PCAP discovery, narrow automated migration patches with validation, semantic detection of custom crypto, binary ML/deep analysis and carefully qualified quantum resource estimation.

28. Final One-Line Pitch

“ECDAT doesn't just tell an organization what cryptography it has; it shows the evidence, connects the assets, explains the migration impact, and verifies that the cryptographic change actually happened.”

This version intentionally excludes the previously discussed demo-website implementation option. The experimental features above are platform capabilities and research extensions, not a separate website demo.